Implementation Log: U-Net to ACU-Net

Step 00 — Baseline Architecture Audit

Status: Source inspected; baseline documented. Runtime verification pending.

Objective

Establish the original U-Net architecture before introducing the attention mechanisms described in the ACU-Net paper.

Baseline implementation

The original model is defined in `UNet/model.py`.

- **Framework:** PyTorch
- **Architecture:** 2D encoder–decoder U-Net
- **Input channels:** 4
- **Input spatial dimensions:** 240 × 240
- **Encoder channels:** 16, 32, 64, 128
- **Bottleneck channels:** 256
- **Decoder channels:** 128, 64, 32, 16
- **Output channels:** 1
- **Convolution block:** Two 3 × 3 convolutions, each followed by batch normalization and ReLU
- **Downsampling:** 2 × 2 max pooling with stride 2
- **Upsampling:** 2 × 2 transposed convolutions with stride 2
- **Skip connections:** Direct concatenation of encoder and decoder features
- **Output activation:** None; the model returns raw logits

Baseline forward pass

1. Process the input through four encoder blocks.
2. Apply max pooling between encoder stages.
3. Process the deepest features through the bottleneck.
4. Upsample the bottleneck features.
5. Concatenate the upsampled features with their corresponding encoder features.
6. Process each concatenated tensor through a decoder block.
7. Apply a 1 × 1 convolution to produce a one-channel segmentation logit map.

Baseline tensor dimensions

For input shape `(1, 4, 240, 240)`, the expected output shape is `(1, 1, 240, 240)`.

The spatial resolutions across the encoder and bottleneck are 240 × 240, 120 × 120, 60 × 60, 30 × 30, and 15 × 15. The decoder restores the original resolution.

Existing limitations relative to the intended ACU-Net

1. No explicit spatial attention module is implemented.
2. No explicit channel attention module is implemented.
3. Encoder–decoder skip connections are not attention-gated.
4. The model architecture alone does not establish whether preprocessing, loss, training, or evaluation matches the paper.

Modification record

No model code has been modified during this step.

Verification

The source includes a test using a random input tensor and checks the resulting output shape. This test is present in the code, but its successful execution has not yet been verified in this audit.

Next step

Inspect the relevant ACU-Net paper sections and map each proposed architectural modification to the baseline before implementing attention modules.

Step 1 — Spatial Attention Module: Implementation and Verification

1. Objective

Implemented a standalone Spatial Attention module in `models/acu_net.py` for the ACU-Net research project.

The module generates a spatial attention map from an input feature tensor and multiplies that map by the original tensor to refine its feature activations.

2. Implementation

The module performs the following operations:

1. Calculates the channel-wise average map.
2. Calculates the channel-wise maximum map.
3. Concatenates both maps along the channel dimension.
4. Applies a convolution with a kernel size of 7 × 7.
5. Applies sigmoid to produce attention weights between 0 and 1.
6. Multiplies the attention map by the original input tensor.

A separate test was used to verify the module's behavior.

3. Verification Setup

The test used a randomly generated input tensor with the shape:

```
(2, 64, 32, 32)
```

Here, the batch size was 2, the number of channels was 64, and the spatial dimensions were 32 × 32.

4. Test Results

Verification	Expected result	Actual result
Output shape	`(2, 64, 32, 32)`	Passed
Attention map shape	`(2, 1, 32, 32)`	Passed
Attention weights	Values between 0 and 1	Min: 0.2060; Max: 0.7626
Input gradient	Available after backpropagation	Passed
Convolution weight gradient	Available after backpropagation	Passed

Overall result: All Spatial Attention tests passed.

5. Interpretation

The module preserves the input tensor's shape while producing a single-channel spatial attention map. The attention weights are within the expected sigmoid range, and the gradient checks confirm that the tested operations support backpropagation.

6. Limitations

The module has been verified using synthetic input data. It has not yet been evaluated on MRI images, integrated into the complete ACU-Net architecture, or assessed for segmentation performance.

The exact correspondence between the implementation and every architectural detail in the research paper still needs to be checked.

7. Next Step

Implement and independently test the Channel Attention module before integrating the attention components into the complete ACU-Net.

Step 2 — Channel Attention Module: Implementation and Verification

1. Objective

Implemented a standalone Channel Attention module in `models/acu_net.py`.

The module learns an importance weight for each feature channel using global average pooling and a bottleneck multilayer perceptron (MLP).

2. Implementation

The module performs the following operations:

1. **Global Average Pooling:** Summarizes each channel by averaging its spatial activations.
2. **Reshaping:** Converts the pooled tensor into a two-dimensional tensor suitable for the MLP.
3. **Bottleneck MLP:** Uses two linear layers, with a ReLU activation between them, to learn channel relationships.
4. **Sigmoid Activation:** Produces channel weights between 0 and 1.
5. **Reshaping and Multiplication:** Reshapes the weights to `(B, C, 1, 1)` and multiplies them by the original input tensor.

The reduction ratio is 16, and the hidden dimension is calculated as `max(channels // reduction, 1)`.

3. Verification Setup

The test used a random input tensor of shape `(2, 64, 32, 32)`, with a reduction ratio of 16.

4. Test Results

Verification	Actual result	Status
Output shape	`(2, 64, 32, 32)`	Passed
Channel weight shape	`(2, 64)`	Passed
Attention weight range	0.3790–0.6241	Passed
Input gradient	Available	Passed
First linear layer gradient	Available	Passed
Second linear layer gradient	Available	Passed

Overall result: All Channel Attention tests passed.

5. Interpretation

The module generates 64 channel weights for each input sample and applies each weight across the corresponding feature channel's spatial dimensions. The output preserves the original tensor shape.

The gradient checks confirm that the tested computation supports backpropagation through the input and both MLP linear layers.

6. Limitations

Verification was performed on synthetic tensors rather than MRI data. The module has not yet been integrated into the complete ACU-Net or evaluated for segmentation performance.

Its exact correspondence to the research paper's implementation must still be checked.

7. Next Step

Implement and independently test the attention-gated skip connection, then integrate the components into the complete architecture.

Step 3 — Attention Gate: Implementation and Verification

1. Objective

Implemented a standalone `AttentionGate` class in `models/acu_net.py` to filter encoder features using information from decoder features.

2. Implementation

The gate performs the following operations:

1. **Encoder projection:** Projects encoder features into an intermediate feature space, treated as key vectors.
2. **Decoder projection:** Projects decoder features into the same intermediate space, treated as query vectors.
3. **Similarity calculation:** Computes the channel-wise dot product between query and key vectors at corresponding spatial locations.
4. **Scaling:** Divides the similarity scores by the square root of the intermediate channel count.
5. **Sigmoid activation:** Converts the scores into attention weights between 0 and 1.
6. **Feature gating:** Multiplies the attention map by the original encoder features.

The gate returns the filtered encoder features. It does not perform the final concatenation with decoder features; that will happen during full architecture integration.

3. Verification Setup

The test used synthetic tensors with the following shapes:

- Encoder features: `(2, 64, 32, 32)`
- Decoder features: `(2, 128, 32, 32)`
- Intermediate channels: `32`

Both inputs had matching spatial dimensions.

4. Test Results

Verification	Actual result	Status
Gated output shape	`(2, 64, 32, 32)`	Passed
Attention map shape	`(2, 1, 32, 32)`	Passed
Attention weight range	0.2473–0.7991	Passed
Encoder gradients	Available	Passed
Decoder gradients	Available	Passed
Encoder projection gradients	Available	Passed
Decoder projection gradients	Available	Passed

Overall result: All Attention Gate tests passed.

5. Interpretation

The gate produces one attention weight per spatial location and uses those weights to filter encoder features without changing their shape. The gradient checks confirm that the tested computation supports backpropagation through both inputs and the projection layers.

6. Limitations and Research Notes

The module was verified using synthetic tensors, not MRI data.

The current implementation compares projected encoder and decoder features at corresponding spatial locations. It is a query/key-style design, but it does not calculate a full spatial query-key attention matrix across all pairs of locations.

The exact attention-gate formulation must be checked against the research paper before claiming exact architectural reproduction.

7. Next Step

Review the paper's attention mechanisms and integrate the three independently tested components into the ACU-Net architecture, while preserving the original U-Net baseline.

Step 4 — Dual Attention Block: Implementation and Verification

1. Objective

Implemented a `DualAttention` class in `models/acu_net.py` to combine Spatial Attention and Channel Attention in parallel.

The block applies both attention mechanisms to the same input feature tensor and adds their outputs.

2. Implementation

The block performs the following operations:

1. **Spatial Attention branch:** Applies the existing `SpatialAttention` module to the input features.
2. **Channel Attention branch:** Applies the existing `ChannelAttention` module independently to the same input features.
3. **Branch combination:** Adds the two branch outputs element-wise.

4. **Output:** Returns the combined feature tensor without changing its dimensions.

If (X) denotes the input features, α the spatial attention map, and β the channel attention weights, the operation is:

$$S = \alpha \odot X$$

Equivalently:

$$Y = (\alpha + \beta) \odot X$$

Here, ⊙ denotes element-wise multiplication with broadcasting.

This additive formulation follows the combination described in Algorithm 1 of the research paper. The existing Spatial Attention and Channel Attention modules were reused without modification.

3. Verification Setup

The test used a synthetic input tensor with shape `(2, 64, 32, 32)` and a channel-attention reduction ratio of 16.

The test independently checked both branches, their combination, output dimensions, and gradient propagation.

4. Test Results

Verification	Actual result	Status
Output shape	`(2, 64, 32, 32)`	Passed
Spatial Attention branch	Executed successfully	Passed
Channel Attention branch	Executed successfully	Passed
Branch combination	Output equals the sum of branch outputs	Passed
Input gradients	Available	Passed
Spatial Attention convolution gradients	Available	Passed
Channel Attention MLP gradients	Both linear layers passed	Passed

Overall result: All Dual Attention tests passed.

5. Interpretation

The block combines spatial and channel-refined feature representations while preserving the original tensor shape. The successful gradient checks confirm that the tested combination supports backpropagation through both attention branches.

Because the two branch outputs are added, the combined attention is not restricted to values between 0 and 1.

6. Limitations

Verification was performed using synthetic tensors. The Dual Attention block has not yet been integrated into the complete ACU-Net, trained on MRI data, or evaluated for segmentation performance.

The implementation follows the additive combination described in Algorithm 1. Other architectural details and their correspondence to the complete paper still require verification.

7. Next Step

Integrate the Dual Attention block and the Attention Gate into the U-Net encoder–decoder architecture, beginning with one stage and verifying tensor dimensions before extending the design across the network.

Step 5: Integrating Dual Attention and Attention Gates into ACU-Net

Objective

Integrate the independently tested attention modules into an encoder–decoder segmentation network while preserving the baseline U-Net channel configuration and skip-connection structure.

Implementation

The ACU-Net implementation contains four encoder stages with 16, 32, 64, and 128 output channels, followed by a 256-channel bottleneck. Dual Attention is applied to the bottleneck features.

Four Attention Gates are used at the decoder skip connections. Each gate receives encoder features and the corresponding upsampled decoder features. The gated encoder features are concatenated with decoder features before the corresponding decoder convolution block.

The final 1×1 convolution produces a single-channel output containing raw segmentation logits.

Verification

The integration test was executed using a synthetic input tensor with shape (1, 4, 240, 240).

Observed output shape: (1, 1, 240, 240).

The test confirmed gradient flow through all four Attention Gates, the Spatial Attention convolution, and the Channel Attention MLP. The complete integration test passed.

Limitations

This test verifies tensor dimensions and gradient flow, not segmentation accuracy. Training, validation on real MRI data, baseline comparison, and evaluation against the paper remain outstanding. The final architecture and attention formulations must also be checked against the paper before claiming faithful reproduction.